



## ***ClosingScript***

This project may appeal to programmers who dislike the significant indentation in CoffeeScript but otherwise like the language for its simplicity and expressiveness.

***ClosingScript*** addresses this sensitive problem by adding closing keywords to close multi-line constructs, making whitespace no longer significant. It is important to understand this only apply to multi-line constructs, everything else, like single-line constructs (one-liners) is unchanged.

***ClosingScript*** is a text preprocessor, not a fork, that regenerates significant indentation to feed the standard CoffeeScript compiler.

A closing keyword consists of the slash followed by the name of its matching multi-line construct.

List of closing keywords:

***/if, /unless, /while, /until, /for, /switch, /loop, /try, /class, /end*** (function)

Advantages:

- Whitespace is not significant.
- Robust code structure.
- Clear visual termination of multi-line constructs.
- Safe copy-pasting.
- Safe auto-formatting.

Disadvantages:

- Slightly more verbose syntax.

## General syntax

The syntax of a multi-line construct is identical to standard CoffeeScript except for the addition of a closing keyword. The list below is not exhaustive and is provided to give a general idea.

```
functionName = (parameters) -> | =>
  code
/end [optional following code]          # examples of following code: /end, timeout
                                         /end )
```

```
do ->
  code
/end
```

```
[variable =] if condition
  code
[else if condition
  code]
[else
  code]
/if
```

```
[variable =] for iterable [guard clauses]
  code
/for
```

```
[variable =] while condition [guard clauses]
  code
/while
```

```
[variable =] loop
  code
/loop
```

```
[variable =] switch [variable]
  when
    code
  [when condition then code]
  [else
    code]
/switch
```

```
class name
  code
/class
```

```
try
  code
[catch ...
  code ]
[finally ...
  code ]
/try
```

## Usage

The preprocessor outputs to *stdout*. To get JavaScript, the standard CoffeeScript compiler must be installed.

Preprocessor and compiler errors match **ClosingScript** sources at the line number level for easier debugging.

Assuming that your **ClosingScript** source file `<filename>.coffee` is in the current directory.

To convert and compile to JavaScript in one step:

```
node <path>/closing.js <filename>.coffee | coffee -s -c > <filename>.js
```

To get a converted copy of a source:

```
node <path>/closing.js <filename>.coffee > <filename>.converted.coffee
```

To get a formatted copy of a source (only reindented consistently):

```
node <path>/closing.js -f <filename>.coffee > <filename>.formatted.coffee
```

## Usage with the Geany code editor

In Linux, this command can be added to the Build menu of the Geany code editor. It converts and compiles in one step, with errors falling back at guilty lines in the editor window, as well as error messages and error codes from the preprocessor and the compiler in the built-in 'Compiler' message window. This conveniently makes the whole process as a single compiling step to the user.

```
set -o pipefail; bun <path>/closing.js "%f" | coffee -s -c > "%e".js 2>>(sed 's/[stdin\]/%f/' >&2)
```